

Task:

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – FUNCTION CALLING & STRUCTURED OUTPUT TASK

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO2 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Function Calling & Structured Output Task
Weightage:	40% of CIA	Total Marks:	1 * 100 = 100 (1 Question1)
CO2 Statement:	<i>Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)</i>		
Student Name:	Sk Abdul Azeez	Reg. No.:	192472015
Section / Batch:	CSE-AI	Date:	04-08-2026

Instructions to Students · Answer 1

question. Total: 100 Marks.

- Analyze the given scenario and identify the appropriate function(s), tool(s), parameters, and structured output required to solve the task.
- Design a valid function-calling schema with appropriate function names, parameters, data types, required fields, and descriptions based on the given requirements.
- Implement the function-calling workflow demonstrating the complete process: User Query → LLM → Function/Tool Call → Function Execution → Structured Response.
- Ensure that the generated output strictly follows the defined structured format or JSON schema and contains valid, relevant, and consistent information.
- Handle appropriate edge cases such as missing parameters, invalid inputs, ambiguous requests, incorrect data types, or unavailable information, and provide suitable responses without producing invalid function calls.
- Demonstrate the working implementation using suitable test cases, including normal inputs and at least one edge-case scenario.
- Clearly document the designed schema by explaining the purpose of each function, its parameters, data types, required fields, expected input, and expected output.
- Briefly justify the design decisions and explain how function calling and structured output improve the reliability, consistency, and usability of the LLM-based application. **Submission Requirements Students should submit:**

-
- **Source code / Jupyter Notebook**
- **Function-calling schema**
- **Structured output/JSON schema**
- **Execution screenshots**
- **Test cases and results**
- **Brief documentation explaining the schema and function-calling workflow**

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: _____

SECTION A — FUNCTION CALLING & STRUCTURED OUTPUT TASK

Task:

4. E-Commerce Product Assistant

An online store wants an AI assistant that can search products based on category, price, and brand.

Task: Design a `search_products()` function with appropriate parameters and structured output. Demonstrate how the assistant handles a query with missing product information.

Solution:

1. Problem Analysis

An online store needs an AI assistant that helps customers search for products based on **category, price, and brand**. Sometimes users may not provide all the required information, so the assistant should ask follow-up questions instead of returning incorrect results.

2. Function Design

```
def search_products(category=None, brand=None, min_price=None, max_price=None):  
    pass
```

Parameters

Parameter	Type	Description
-----------	------	-------------

category	String	Product category (e.g., Mobile, Laptop)
----------	--------	---

brand	String	Product brand (e.g., Samsung, Apple)
-------	--------	--------------------------------------

min_price	Integer	Minimum price
-----------	---------	---------------

max_price	Integer	Maximum price
-----------	---------	---------------

3. Structured Output

Task:

```
{  
  
  "category": "Mobile",  
  
  "brand": "Samsung",  
  
  "price_range": {  
  
    "min": 20000,  
  
    "max": 40000  
  
  },  
  
  "status": "Success",  
  
  "products": [  
  
    {  
  
      "name": "Samsung Galaxy A56",  
  
      "price": 32999  
  
    },  
  
    {  
  
      "name": "Samsung Galaxy M56",  
  
      "price": 28999  
  
    }  
  
  ]  
  
}
```

4. Example Query**User Query**

"Show Samsung mobiles between ₹20,000 and ₹40,000."

Assistant Output

Task:

```
{  
  
  "category": "Mobile",  
  
  "brand": "Samsung",  
  
  "price_range": "₹20,000 - ₹40,000",  
  
  "products": [  
  
    "Samsung Galaxy A56",  
  
    "Samsung Galaxy M56"  
  
  ]  
}
```

5. Handling Missing Product Information**User Query**

"Show me mobiles."

Missing Information

- Brand ✕
- Price Range ✕

Assistant Response

"Sure! Which brand are you looking for (Samsung, Apple, Vivo, etc.)? Also, what is your budget or price range?"

Another Example**User Query**

"I want an Apple product."

Missing Information

- Category ✕

Task:

- Price Range ✕

Assistant Response

"Are you looking for an iPhone, iPad, MacBook, or Apple Watch? Please also mention your budget."

6. Expected Outcome

- Searches products accurately using category, brand, and price.
- Returns structured JSON output.
- Detects missing information.
- Asks follow-up questions before searching.
- Improves the customer shopping experience.

7. Conclusion

The `search_products()` function enables an AI-powered e-commerce assistant to perform intelligent product searches using category, brand, and price filters. When the user's query is incomplete, the assistant requests the missing details instead of guessing, resulting in more accurate recommendations and a better shopping experience.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Schema Design	30%	Correctly designs a function-calling schema and structured (JSON) output format	Mostly correct schema with minor issues	Schema has significant errors	Schema is fundamentally incorrect or missing
Output Validity	25%	LLM reliably produces valid structured output matching the schema	Mostly valid output with occasional errors	Output is frequently invalid or inconsistent	Output rarely or never matches the schema
Edge-Case Handling	20%	Handles edge cases (missing fields, ambiguous input) gracefully	Handles common cases well	Handles only the simplest cases	Fails on basic cases
Function-Call Flow Demo	15%	Clear demo of the function-call to structured-response flow	Demo covers the main flow	Demo is incomplete	No functional demo presented
Schema Documentation	10%	Well-documented schema design rationale	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Schema Design(30%)	Output Validity (25%)	Edge-Case Handling (20%)	Function-Call Flow Demo(15%)	Schema Documentation (10%)	Total Marks (100)
Q1						
Overall Total (100)						
Faculty In-charge		Course Coordinator		Head of Department		
Signature: _____		Signature: _____		Signature: _____		
Date: _____		Date: _____		Date: _____		